

Fondamenti di Computer Grafica - Relazione

Emanuele Molinari
S5797893

Indice

Introduzione	2
Giocatore (Tappe 1-2)	3
Nemici (Tappe 3-4-5-6)	4
Schermate di inizio e game over, GUI (Tappe 7-8-9)	7
Modifiche al game over, effetti sonori (Tappe 10-11)	9
Schermata di pausa, scudi, velocità incrementale nemici (Tappa 12)	11
Ricariche scudi, proiettile bonus, riorganizzazione file (Tappe 13-14)	13
Notifica punteggi (Tappa 15)	16

Introduzione

Il progetto si basa sul gioco "Space Invaders". Le texture non sono quelle dell'originale e gli scudi, disponibili in quantità limitata, devono essere attivati dal giocatore poco prima di essere colpito. Eliminando i nemici è possibile che questi rilascino delle ricariche che, se il giocatore raccoglie, aggiungono uno scudo disponibile.

Eliminando il nemico bonus, una nave che ha una certa probabilità di passare sopra ai nemici, il giocatore guadagna automaticamente un colpo bonus (nuke) che colpisce i nemici in un'area di $3 \cdot 3$. Il giocatore può avere un solo colpo bonus a disposizione alla volta, e nel caso in cui lo utilizzi contro un'ulteriore nave bonus, la eliminerà in un colpo solo invece di 3. Se la nuke è disponibile, comparirà un alert lampeggiante nella parte bassa dello schermo.

Superando ogni livello si guadagna una vita e uno scudo. Le risorse sono state inserite tramite embedding di file `hpp`.

Di seguito sono riportate le aggiunte principali di tappa in tappa. L'organizzazione dei file cambia notevolmente, e con il proseguire delle tappe molte funzioni descritte vengono spostate e rivisitate.

Giocatore (Tappe 1-2)

Tappa 1 Le risorse sono gestite in State. E' stata inizializzata la finestra di gioco, adattandola alla risoluzione del desktop dell'utente e impostando il limite a 60 FPS. Lo sfondo è stato scalato proporzionalmente per riempire l'intero schermo, mentre lo sprite della navicella è stato centrato rispetto alla propria origine, scalato e posizionato staticamente nella parte bassa e centrale della finestra:

```
State() :
    background(spacebackground_jpg, spacebackground_jpg_len),
    background_sprite(background),
    player(player_png, player_png_len),
    player_sprite(player)
{
    //recupero desktop e creazione finestra
    sf::VideoMode desktop = sf::VideoMode::getDesktopMode();
    window.create(sf::VideoMode(desktop.size.x, desktop.size.y), "Space Invaders");
    window.setFramerateLimit(60);

    //sfondo di dimensione dello schermo
    double background_scale_x = (static_cast<float>(desktop.size.x) / background.getSize().x);
    double background_scale_y = (static_cast<float>(desktop.size.y) / background.getSize().y);
    background_sprite.setScale(sf::Vector2f(background_scale_x, background_scale_y));

    //sposta origine di player al centro dello sprite
    float player_centro_x = static_cast<float>(player.getSize().x) / 2.0;
    float player_centro_y = static_cast<float>(player.getSize().y) / 2.0;
    player_sprite.setOrigin(sf::Vector2f(player_centro_x, player_centro_y));

    player_sprite.setScale(sf::Vector2f(0.5, 0.5));
    player_sprite.setPosition(sf::Vector2f(static_cast<float>(desktop.size.x) / 2.0, static_cast<float>(desktop.size.y) * 0.8));
}
```

E' stato implementato il game loop principale, che separa l'intercettazione degli eventi, e successivamente anche le update, dal rendering doGraphics.

Tappa 2 Ora il giocatore può muoversi lungo l'asse orizzontale. L'input dei tasti viene gestito nella update. Quando il giocatore preme spazio, se è scaduto il cooldown, un nuovo proiettile viene aggiunto al vettore dei proiettili. L'incremento della posizione dei proiettili avviene attraverso un ciclo for che scorre quelli presenti nel relativo vettore.

Nemici (Tappe 3-4-5-6)

Tappa 3 Sono stati aggiunti i nemici, in particolare:

- Struct Enemy: è stata creata la struttura Enemy e un'enumerazione enemyType per diversificarli leggermente in estetica e comportamento futuro.
- Animazione basilare tramite spritesheet: è stato implementato il metodo animate(). Utilizzando sf::IntRect, la funzione prima definisce i frame e poi gli alterna basandosi su un intervallo di tempo governato da un sf::Clock.
- Generazione a griglia: nello state è stato inserito il posizionamento dei nemici, basato su cicli annidati (righe e colonne) per generare la griglia. I nemici vengono posizionati calcolando a ogni iterazione la posizione riga-colonna $posY = startY + (i * distY)$, analogamente per le X (colonne), assegnando tipi e texture differenti in base alla riga di appartenenza. Il vettore che contiene i nemici è monodimensionale, in quanto la posizione dei nemici viene salvata nel relativo sprite. L'annidamento serve solamente a calcolare di volta in volta a quale riga-colonna associare la posizione dello sprite.

```
//posizione nemici
float screenWidth = static_cast<float>(desktop.size.x);
float screenHeight = static_cast<float>(desktop.size.y);

float distX = 300; //distanze tra nemici
float distY = 200;

float gridWidth = (columns - 1) * distX; //dimensioni griglia
float gridHeight = (rows - 1) * distY;

float startX = (screenWidth - gridWidth) / 2; //posizionamento effettivo griglia
float startY = screenHeight * 0.15;

for(int i = 0; i < rows; i++) {
    for(int j = 0; j < columns; j++) {
        float posX = startX + (j * distX);
        float posY = startY + (i * distY);

        if(i == 2){
            Enemy en(enemy1_texture, Type1);
            en.sprite.setPosition(sf::Vector2f(posX, posY));
            en.sprite.setScale(sf::Vector2f(0.8, 0.8));
            enemies.push_back(en);
        }
        else if(i == 1){
            Enemy en(enemy2_texture, Type2);
            en.sprite.setPosition(sf::Vector2f(posX, posY));
            en.sprite.setScale(sf::Vector2f(1.2, 1.2));
            enemies.push_back(en);
        }
        else {
            Enemy en(enemy3_texture, Type3);
            en.sprite.setPosition(sf::Vector2f(posX, posY));
            enemies.push_back(en);
        }
    }
}
```

Tappa 4

- La logica di inizializzazione del giocatore e dei nemici è stata spostata in delle funzioni dedicate `initPlayer` e `initEnemies`, chiamate poi nello `State`. Anche la logica di movimento e sparo del giocatore è stata suddivisa in due funzioni ausiliarie di `update`.
- Struct `EnemyBullets` con relativa `update` per gestire gli spari dei nemici, analogamente a quanto fatto con `Player`. Per il momento solo i nemici della riga più bassa possono sparare tramite il booleano `emptyFront`, la frequenza degli spari è gestita tramite `rand()` e nemici di tipi diversi hanno cooldown diversi: più sono lontani dal giocatore e più la frequenza di sparo è maggiore.

```
//proiettili nemici
void updateEnemyBullets(State& gs) {
    for(auto& enemy : gs.enemies) {
        int shoot = rand() % 100;
        if(enemy.emptyFront) {
            switch(enemy.type) {
                case Type1:
                    if(shoot < 2 && enemy.enemyBullet_clock.getElapsedTime().asSeconds() >= 2.8){
                        gs.enemyBullets.push_back(enemyBullet(gs.enemyBullet_texture, enemy.sprite.getPosition()));
                        enemy.enemyBullet_clock.restart();
                    }
                    break;
                case Type2:
                    if(shoot < 2 && enemy.enemyBullet_clock.getElapsedTime().asSeconds() >= 0.75){
                        gs.enemyBullets.push_back(enemyBullet(gs.enemyBullet_texture, enemy.sprite.getPosition()));
                        enemy.enemyBullet_clock.restart();
                    }
                    break;
                case Type3:
                    if(shoot < 2 && enemy.enemyBullet_clock.getElapsedTime().asSeconds() >= 0.5){
                        gs.enemyBullets.push_back(enemyBullet(gs.enemyBullet_texture, enemy.sprite.getPosition()));
                        enemy.enemyBullet_clock.restart();
                    }
                    break;
            }
        }
    }

    //scorrimto proiettili nemici
    for(auto& enemyBullet : gs.enemyBullets) {
        enemyBullet.pos.y += enemyBullet.speed;
        enemyBullet.sprite.setPosition(enemyBullet.pos);
    }
}
```

- Mediante `updateEnemies` è stato gestito il movimento dei nemici, inizialmente sempre alla stessa velocità indipendentemente dal loro numero.

```
//spostamento nemici
void updateEnemies(State& gs) {
    if(gs.move_clock.getElapsedTime().asSeconds() >= 1.0) {
        if(!gs.enemies.empty()) {
            float dist = 50;
            bool edge = false;

            float minX = gs.enemies[0].sprite.getPosition().x; //trova estremi
            float maxX = minX;
            for(auto& enemy : gs.enemies) {
                float x = enemy.sprite.getPosition().x;
                if(x > maxX) maxX = x;
                if(x < minX) minX = x;
            }

            float windowHeight = static_cast<float>(gs.window.getSize().x);

            if((maxX + dist >= windowHeight - 170.0 && gs.right_dir) || (minX - dist <= 170.0 && !gs.right_dir)) {
                edge = true;
            }

            if(edge) {
                gs.right_dir = !gs.right_dir;
                for(auto& enemy : gs.enemies) {
                    enemy.sprite.move(sf::Vector2f(0.0, 30.0)); //nemici scendono
                    enemy.animate(); //sprite animaz
                }
            }
            else {
                for(auto& enemy : gs.enemies) {
                    if(gs.right_dir) enemy.sprite.move(sf::Vector2f(dist, 0.0));
                    else enemy.sprite.move(sf::Vector2f(-dist, 0.0));
                    enemy.animate();
                }
            }
            gs.move_clock.restart();
        }
    }
}
```

Tappa 5

- emptyFront è stato sostituito dal metodo isFrontEnemy all'interno della struttura Enemy. Assegnando a ogni nemico un identificativo di colonna (col), la funzione verifica se esistono alleati ancora vivi con una coordinata Y maggiore: se sì il nemico non spara.
- Gestione delle collisioni: È stata introdotta la funzione updatePlayerBulletsCollisions, che verifica le intersezioni tramite getGlobalBounds().findIntersection() dei proiettili del giocatore e dei nemici attivi. Al rilevamento dell'impatto, il nemico viene segnato come morto (isAlive = false) e il proiettile disattivato istantaneamente spostandone la Y a -500. I nemici eliminati, così come i proiettili ormai sparati fuori dallo schermo, vengono eliminati tramite erase.

```
//collisioni proiettile giocatore
void updatePlayerBulletsCollisions(State& gs) {
    for(auto& playerBullet : gs.playerBullets) {
        sf::FloatRect playerBulletBounds = playerBullet.sprite.getGlobalBounds();

        for(auto& enemy : gs.enemies) {
            if(enemy.isAlive) {
                sf::FloatRect enemyBounds = enemy.sprite.getGlobalBounds();

                if(playerBulletBounds.findIntersection(enemyBounds).has_value()) {
                    enemy.isAlive = false;

                    playerBullet.pos.y = -500;
                    break;
                }
            }
        }
    }

    for(int i = 0; i < gs.enemies.size(); i++) {
        if(!gs.enemies[i].isAlive) gs.enemies.erase(gs.enemies.begin() + i);
    }

    for(int i = 0; i < gs.playerBullets.size(); i++) {
        if(gs.playerBullets[i].pos.y < 0.0) gs.playerBullets.erase(gs.playerBullets.begin() + i);
    }
}
```

Tappa 6

- updateEnemyBulletsCollisions: analogamente a sopra, è stata aggiunta la logica di gestione delle collisioni tra il giocatore e i proiettili nemici. Sono state aggiunte anche le vite rimanenti nella struct Player, e un momentaneo GameOver che consiste nella terminazione del programma.
- Struct Explosion: è stato introdotto uno sprite delle esplosioni relative ai proiettili che colpiscono il giocatore e i nemici. Quando un proiettile colpisce il bersaglio, viene istanziata un'esplosione esattamente alle coordinate dell'entità colpita, inserendola nel vettore gs.explosions. Le funzioni di aggiornamento delle collisioni verificano il timer di ogni esplosione attiva. Superata la soglia, l'oggetto viene rimosso.

Schermate di inizio e game over, GUI (Tappe 7-8-9)

Tappa 7

- E' stata introdotta la base della schermata di GameOver (qui è solamente lo sfondo vuoto), che compare una volta che le vite del giocatore arrivano a -1, attraverso il booleano gameOver e updateGameOver che, se gameOver = true, ripulisce i vettori di oggetti temporanei (esplosioni, proiettili) e il gioco si blocca.
- Se durante gameOver = true il giocatore preme invio, attraverso una handle, viene riavviata la partita con un ripristino dei nemici, vite, punti.

Tappa 8

- Struct Start: stata introdotta una schermata iniziale con il titolo del gioco e la scritta lampeggiante "Premi invio per giocare". La funzione updateCaption() introduce l'effetto utilizzando un sf::Clock dedicato (effect_clock): se i secondi trascorsi sono pari, il testo diventa trasparente, bianco altrimenti.

```
struct Start {
    sf::Font font;
    sf::Text title;
    sf::Text caption;
    sf::Clock effect_clock; //clock per effetto

    Start() {
        title(font);
        caption(font);
    }

    {
        font.openFromMemory(font_ttf, font_ttf_len);

        //title
        title.setString("Space Invaders");
        title.setCharacterSize(256);

        sf::FloatRect bounds = title.getLocalBounds();
        title.setOrigin(sf::Vector2f(bounds.size.x / 2, bounds.size.y / 2));

        title.setFillColor(sf::Color::White);
        title.setPosition(sf::Vector2f(sf::VideoMode::getDesktopMode().size.x / 2.0, (sf::VideoMode::getDesktopMode().size.y / 2.0) * 0.5));

        //max di premere invio
        caption.setString("Premi invio per giocare");
        caption.setCharacterSize(64);

        sf::FloatRect cbounds = caption.getLocalBounds();
        caption.setOrigin(sf::Vector2f(cbounds.size.x / 2, cbounds.size.y / 2));

        caption.setFillColor(sf::Color::White);
        caption.setPosition(sf::Vector2f(sf::VideoMode::getDesktopMode().size.x / 2.0, (sf::VideoMode::getDesktopMode().size.y / 2.0) * 0.8));
    }

    void updateCaption() {
        if(static_cast<int>(effect_clock.getElapsedTime().asSeconds()) % 2 == 0) {
            caption.setFillColor(sf::Color::Transparent);
        }
        else {
            caption.setFillColor(sf::Color::White);
        }
    }
}
```

- Aggiunta una prima versione di un'interfaccia grafica Lives_ui, successivamente rinominata in Ui. Qui contiene solo le vite e il contatore dei punti (tappa 9), successivamente conterrà anche

il contatore degli scudi disponibili e l'alert per il colpo bonus. La Ui si aggiorna attraverso la relativa update.

```
struct Lifes_ui {
    sf::Font font;
    sf::Text livesText;

    Lifes_ui() :
        livesText(font)
    {
        font.openFromMemory(font_ttf, font_ttf_len);

        livesText.setString("Vite: 3");
        livesText.setCharacterSize(64);
        livesText.setFillColor(sf::Color::White);
        livesText.setPosition(sf::Vector2f(sf::VideoMode::getDesktopMode().size.x * 0.81, sf::VideoMode::getDesktopMode().size.y * 0.89));
    }

    void update(int playerLives) {
        livesText.setString("Vite: " + std::to_string(playerLives));
    }

    void draw(sf::RenderWindow window) const {
        window.draw(livesText);
    }
};
```

Tappa 9

- Nella struct Enemy è stato aggiunto il punteggio, diverso per ogni tipo di nemico e che viene assegnato al giocatore a ogni eliminazione in `updatePlayerBulletsCollisions`.
- Alla Ui è stata aggiunta una stringa riportante il punteggio corrente, che si aggiorna analogamente alla precedente update.
- E' stata implementata effettivamente la schermata di GameOver mediante la struct End, in cui compare "GameOver" e il messaggio di premere invio per riavviare la partita, con lo stesso effetto intermittente utilizzato nella schermata Start.

Modifiche al game over, effetti sonori (Tappe 10-11)

Tappa 10

- Progressione livello: tramite `updateLevel` si controlla il vettore dei nemici, se è vuoto il giocatore guadagna una vita extra (`playerLives++`), vengono puliti i vettori dei proiettili e dei giocatori e vengono chiamate `initPlayer` e `initEnemies` per resettare il livello. Il contatore dei livelli non è ancora presente.
- Ora la sconfitta non dipende solo dall'esaurimento delle vite, ma anche dalla posizione dei nemici. Tramite un ciclo `for`, viene individuata la coordinata Y massima, ossia il nemico più in basso. Se questa supera una certa altezza dello schermo, il flag di sconfitta (`lost = true`) si attiva, causando il `gameOver`.

```
void updateGameOver(State& gs) {
    bool lost = false;

    //se i nemici si avvicinano troppo
    if(!gs.enemies.empty()) {
        for(auto& enemy : gs.enemies) {
            float maxY = enemy.sprite.getPosition().y;
            if(maxY > sf::VideoMode::getDesktopMode().size.y * 0.6) lost = true;
        }
    }

    if(gs.playerLives < 0){
        lost = true;
    }

    if(lost) {
        gs.gameoverTransition = true;
        gs.gameoverTransition_clock.restart();
    }
}
```

- In precedenza, la morte azzerava istantaneamente lo schermo. Nella struct `State` sono stati aggiunti `bool gameOverTransition` e `sf::Clock gameOverTransition_clock`. Quando si innesca la condizione di sconfitta, il gioco entra ora in una fase di transizione. La funzione `update()` principale intercetta questo stato e impone un blocco di 0.5 secondi. Durante questo periodo la logica si ferma.

- Adesso se le vite del giocatore sono minori di 0, viene posto a true il booleano lost, viene impostato a true gameOverTransition e, una volta passati 0.5 secondi, gameOver viene impostato a true e quindi nella doGraphics viene disegnata la schermata di Gameover.

Tappa 11 In questa si è diviso il codice in vari file:

- main.cpp: contiene il main loop, il doGraphics, le funzioni di handle e la update principale, in cui vengono chiamate le ausiliarie
- entities.hpp: contiene le struct relative ai nemici, il giocatore, i proiettili e le esplosioni con i relativi metodi
- interfaces: contiene le struct delle varie schermate come Start, End e la Ui
- state.hpp: contiene struct State e le firme delle update ausiliarie
- stateUpdate.cpp: contiene implementazione delle update ausiliarie, i costruttori e metodi di interfaces e state

L'organizzazione dei file cambierà ulteriormente, vedere l'ultima tappa per quella definitiva.

- Sono stati anche aggiunti effetti sonori alle esplosioni e agli spari del giocatore, senza aggiungerli agli spari dei nemici per evitare troppa confusione. Analogamente alle texture, si è utilizzato l'embedding di file.hpp, ad eccezione della soundtrack di sottofondo che è stata passata tramite argomento al main e metodo playMusic.

```
int main(int argc, char* argv[]) {
    std::string audioDir = "./media/audio"; //se non scrivo da terminale, questo è default
    if (argc >= 2) audioDir = argv[1];

    State gs;
    srand(time(NULL));
    gs.audioDir = audioDir; //passo a state la cartella per cercare audio e usare la funzione playMusic

    gs.playMusic("soundtrack.mp3"); //riproduce audio con var soundtrack
    while (gs.window.isOpen()) {
        gs.window.handleEvents([&](const auto &event) {
            handle(event, gs);
        });

        update(gs);

        doGraphics(gs);
    }
}
```

```
void State::playMusic(const std::string& trackName) { //utilizzo playMusic 1 volta nel main, 1 volta al restart e al gameover faccio soundtrack.stop()
    std::string path = audioDir + "/" + trackName;
    soundtrack.openFromFile(path);
    soundtrack.stop(); //se c'era qualcosa prima, lo ferma
    soundtrack.setLooping(true);
    soundtrack.setVolume(50.0);
    soundtrack.play();
}
```

Schermata di pausa, scudi, velocità incrementale nemici (Tappa 12)

Schermata di pausa E' stata aggiunta una schermata di pausa, che appare e ferma la partita premendo esc. Sono presenti 3 opzioni: riprendi, esci (termina l'esecuzione) e riavvia partita (aggiunta successivamente). Le opzioni sono selezionabili con le frecce oppure con il mouse. L'implementazione è stata realizzata in interfaces, assieme a quelle delle altre schermate. La navigazione è gestita dai metodi della struct Pause, mentre nelle handle del main si collegano i metodi di Pause agli input (mouse o tastiera) ricevuti per navigare e selezionare.

```
void Pause::up() {
    if(selectedCaptionIndex - 1 >= 0) {
        captions[selectedCaptionIndex].setFillColor(sf::Color(128,128,128));
        selectedCaptionIndex--;
        captions[selectedCaptionIndex].setFillColor(sf::Color::White);
    }
    else down();
}

void Pause::down() {
    if(selectedCaptionIndex + 1 < 2) {
        captions[selectedCaptionIndex].setFillColor(sf::Color(128,128,128));
        selectedCaptionIndex++;
        captions[selectedCaptionIndex].setFillColor(sf::Color::White);
    }
    else up();
}

void Pause::mouse(sf::Vector2f mousePos) {
    for(int i = 0; i < 2; i++) {
        if(captions[i].getGlobalBounds().contains(mousePos)) { //se mouse è su scritta
            if(selectedCaptionIndex != i) { //se != i ignora e resta di default
                captions[selectedCaptionIndex].setFillColor(sf::Color(128,128,128)); //scelta precedente
                selectedCaptionIndex = i; //aggiorna scelta corrente
                captions[selectedCaptionIndex].setFillColor(sf::Color::White); //scelta corrente
            }
        }
    }
}
```

Questi metodi permettono di scorrere le voci del menu. Quando richiamati, aggiornano l'indice della selezione e invertono i colori delle scritte (bianco per quella attualmente selezionata). La logica di selezione ciclica gli estremi sarà rivisitata in seguito poichè non più compatibile con l'aggiunta della terza opzione di riavvio. Il metodo mouse riceve in input le coordinate del cursore. Tramite un ciclo, sfrutta la funzione `getGlobalBounds().contains()` per verificare se il cursore si sovrappone all'area di una delle scritte. In caso di collisione, se la voce puntata è diversa da quella attualmente selezionata, il metodo aggiorna l'index e i colori.

```

//movimento mouse pausa
void handle(const sf::Event::MouseMoveEvent &event, State &gs) {
    sf::Vector2f mousePos = gs.window.mapPixelToCoords(sf::Vector2i(event.position.x, event.position.y));
    gs.pause.mouse(mousePos);
}

//click mouse pausa
void handle(const sf::Event::MouseButtonPressed &event, State &gs) {
    sf::Vector2f mousePos = gs.window.mapPixelToCoords(sf::Vector2i(event.position.x, event.position.y));
    if(gs.isPaused && event.button == sf::Mouse::Button::Left) {
        if(gs.pause.captionIndex == gs.pause.selectedCaptionIndex) {
            if(gs.pause.selectedCaptionIndex == 0) {
                gs.isPaused = false;
                gs.soundtrack.play();
            }
            else if (gs.pause.selectedCaptionIndex == 1) gs.window.close();
        }
    }
}

```

Ad ogni spostamento del mouse, l'evento converte le coordinate dei pixel della finestra nelle coordinate assolute del mondo di gioco tramite `mapPixelToCoords()`. Queste coordinate vengono passate costantemente al metodo `mouse()` della schermata di pausa, permettendo la selezione. Se il gioco è in stato di pausa e viene premuto il tasto sinistro del mouse, l'handler verifica prima le coordinate del click. Se queste coincidono con l'area dell'opzione attualmente evidenziata, viene eseguita l'azione corrispondente all'indice: l'indice 0 rimuove lo stato di pausa e fa ripartire la colonna sonora (`gs.soundtrack.play()`), mentre 1 chiude definitivamente la finestra con `gs.window.close()`.

```

if(gs.isPaused) { //navigazione pausa con tastiera
    if(event.code == sf::Keyboard::Key::Up) {
        gs.pause.up();
    }
    if(event.code == sf::Keyboard::Key::Down) gs.pause.down();
    if(event.code == sf::Keyboard::Key::Enter) {
        if(gs.pause.selectedCaptionIndex == 0) { //riprendi
            gs.isPaused = false;
            gs.soundtrack.play();
        }
        else gs.window.close();
    }
}

```

La pressione delle frecce invoca i rispettivi metodi `up()` e `down()`. La pressione del tasto invio funge da tasto di conferma e, analogamente al click del mouse, viene letto il valore di `selectedCaptionIndex` in quell'istante e si esegue l'azione selezionata.

Scudi Sono stati aggiunti gli scudi attivabili dal giocatore, al momento il contatore è assente nell'interfaccia e non possono ancora essere raccolti. E' stata realizzata una struct `shield`, un metodo `updateShield` che attiva lo scudo per una durata di 0.7 secondi, con un cooldown di 1.0. E' stato aggiunto anche un suono che si riproduce al momento dell'impatto di un proiettile nemico con lo scudo.

Velocità incrementale dei nemici Da ora i nemici, mano a mano che diminuiscono, si muovono sempre più velocemente. L'ultimo nemico rimasto ha una velocità molto più alta del normale. Per tenere traccia del numero di nemici qui si utilizza una variabile `enemiesQuantity`, di seguito verrà eliminata e sostituita direttamente dalla `size` del vector contenente i nemici.

Ricariche scudi, proiettile bonus, riorganizzazione file (Tappe 13-14)

Ricariche scudi Sono state aggiunte le ricariche degli scudi, che possono comparire all'eliminazione di un nemico. Il giocatore può raccogliarli per incrementare gli scudi disponibili. Oltre alla struct `ShieldCharger`, in cui viene gestito lo sprite e l'animazione, sono presenti i metodi `update` ausiliari `dropShieldCharger` (qui in `update`, successivamente accorpato alla struct) che viene chiamata in `updatePlayerBulletsCollisions` e regola l'apparizione della ricarica, e `pickShieldCharger` che gestisce la logica di raccolta.

```
//per gestire gli scudi bonus (ausiliaria di updatePlayerBulletsCollisions)
void dropShieldCharger(State& gs, Enemy enemy) {
    float prob = rand() % 100;
    if(prob <= 3.0 && !gs.shieldChargerReleased) {
        gs.shieldChargerReleased = true;
        gs.shieldCharger.sprite.setPosition(enemy.sprite.getPosition());
    }
}

void pickShieldCharger(State& gs) {
    if(gs.shieldChargerReleased) {
        gs.shieldCharger.animate();
        gs.shieldCharger.sprite.move(sf::Vector2f(0, gs.shieldCharger.speed));

        sf::FloatRect shieldChargerBounds = gs.shieldCharger.sprite.getGlobalBounds();
        sf::FloatRect playerBounds = gs.player.sprite.getGlobalBounds();
        if(shieldChargerBounds.findIntersection(playerBounds).has_value()) {
            gs.player.shields++;

            gs.shieldChargerSound.play();
            gs.shieldCharger.sprite.setPosition(sf::Vector2f(0, -500));
            gs.shieldChargerReleased = false;
        }

        else if(gs.shieldCharger.sprite.getPosition().y >= sf::VideoMode::getDesktopMode().size.y) {
            gs.shieldChargerReleased = false;
        }
    }
}
```

Lo sprite appare alla posizione del nemico appena sconfitto, passata come input di `dropShieldCharger` dentro la funzione `updatePlayerBulletsCollisions`. `pickShieldCharger` si occupa di muovere verticalmente la ricarica verso il basso e, se intercetta i bound del giocatore, viene raccolto e incrementa gli scudi. Al momento della raccolta viene riprodotto un effetto sonoro. Infine è stato aggiunto alla `Ui` il contatore degli scudi rimanenti.

Nave e proiettile bonus L'implementazione del colpo bonus (nuke) e della `bonusShip` (qui ancora chiamata `nukeShip`) introduce un colpo ad area e il relativo sistema per ottenerla. La fun-

zione `spawnNukeship()` gestisce la generazione della nave nella parte superiore dello schermo con una probabilità molto bassa (circa 1 su 10.000 frame). Quando appare, la navicella attraversa lo schermo orizzontalmente ed è dotata di 3 vite. Se il giocatore riesce a distruggerla, (`updateNukeShipCollisions()`), viene ricompensato con 50 punti e la ricarica di un colpo speciale incrementando `player.nukes`. La logica di sparo è gestita da `shootPlayerNuke()`. La gestione delle collisioni di `updateNukeCollision()` fa sì che al primo impatto con un nemico, venga calcolata un'area di danno attorno al punto di collisione. Si itera poi sull'intero vettore dei nemici, verificando l'intersezione con quest'area ed eliminando tutte le entità coinvolte. L'esplosione associata è di dimensioni maggiori, e possiede un effetto sonoro. Se la nave bonus è colpita da un colpo bonus, viene distrutta al primo colpo.

```
void updateNukeCollision(State& gs) {
    bool hitTarget = false;
    sf::Vector2f hitPos;
    sf::FloatRect nukeBounds = gs.nuke.sprite.getGlobalBounds();

    //impatto
    for(auto& enemy : gs.enemies) {
        if(enemy.isAlive()) {
            sf::FloatRect enemyBounds = enemy.sprite.getGlobalBounds();

            if(nukeBounds.findIntersection(enemyBounds).has_value()) {
                hitTarget = true;
                hitPos = enemy.sprite.getPosition();

                gs.nukeSound.play();
                gs.nuke.sprite.setPosition(sf::Vector2f(0, -500));
                gs.existsNuke = false;
                break;
            }
        }
    }

    //area esplosione
    if(hitTarget) {
        float damageWidth = 500.0; //dim totali
        float damageHeight = 260.0;
        sf::Vector2f damagePosition(hitPos.x - (damageWidth / 2.0), hitPos.y - (damageHeight / 2.0)); //coordinate angolo alto sx dell'esplosione
        sf::Vector2f damageSize(damageWidth, damageHeight); //raggruppo le dim in un unico oggetto
        sf::FloatRect damageArea(damagePosition, damageSize); //unisce pos iniziale e dimensioni

        int tempCounter = 0; //per far apparire solo 1 esplosione e non 3x3
        for(auto& enemy : gs.enemies) {
            if(enemy.isAlive()) {
                sf::FloatRect enemyBounds = enemy.sprite.getGlobalBounds();
                if(damageArea.findIntersection(enemyBounds).has_value()) {
                    dropShieldCharger(gs, enemy);
                    enemy.isAlive = false;
                    gs.enemiesQuantity--;
                    gs.player.score += enemy.points;

                    tempCounter++;
                    if(tempCounter == 1) {
                        Explosion exp(3.0, gs.assets.explosion_texture, hitPos);
                        gs.explosions.push_back(exp);
                    }
                }
            }
        }
        tempCounter = 0;
    }

    eraseEnemies(gs);
    expireExplosions(gs);
}
```

```
void spawnNukeship(State& gs) {
    float distY = sf::VideoMode::getDesktopMode().size.y * 0.04; //per impostare altezza navicella + controlli sotto
    if(!gs.existsNukeShip) {
        float spawnProb = rand() % 10000;
        if(spawnProb <= 1.0) gs.existsNukeShip = true; //1 su 10k frame circa

        float dirProb = rand() % 100;
        if(dirProb <= 50) {
            gs.nukeship.rightDirection = true;
            gs.nukeship.sprite.setPosition(sf::Vector2f(0.0, distY));
        }
        else {
            gs.nukeship.rightDirection = false;
            gs.nukeship.sprite.setPosition(sf::Vector2f(sf::VideoMode::getDesktopMode().size.x, distY));
        }
        gs.nukeship.lives = 3;
        gs.nukeship.setDirection(gs.nukeship.rightDirection);
    }
    else {
        gs.nukeship.move(gs.nukeship.rightDirection);

        if(!gs.nukeship.rightDirection && gs.nukeship.sprite.getPosition() == sf::Vector2f(0.0, distY)) {
            gs.existsNukeShip = false;
        }
        else if(gs.nukeship.rightDirection && gs.nukeship.sprite.getPosition() == sf::Vector2f(sf::VideoMode::getDesktopMode().size.x, distY)) {
            gs.existsNukeShip = false;
        }
    }
}
```

Il booleano `existNukeShip` al momento appartiene a `State`, ma verrà poi incorporato alla `Struct` insieme alla funzione `spawnNukeShip`. La nave possiede poi un altro booleano `rightDirection` che serve a definirne la direzione, e successivamente quale spritesheet utilizzare.

```
void updateNukeShipCollisions(State& gs) {
    if(gs.existsNukeShip) {
        sf::FloatRect nukeshipBounds = gs.nukeship.sprite.getGlobalBounds();
        for(auto& bullet : gs.playerBullets) {
            sf::FloatRect playerBulletBounds = bullet.sprite.getGlobalBounds();

            if(playerBulletBounds.findIntersection(nukeshipBounds).has_value()) {
                gs.nukeship.lives--;

                bullet.sprite.setPosition(sf::Vector2f(0, -500));
                Explosion exp(0.5, gs.assets.explosion_texture, gs.nukeship.sprite.getPosition());
                gs.explosions.push_back(exp);
            }
        }

        sf::FloatRect nukeBounds = gs.nuke.sprite.getGlobalBounds();
        if(gs.existsNuke && nukeBounds.findIntersection(nukeshipBounds).has_value()) {
            gs.nukeship.lives = 0;

            gs.nuke.sprite.setPosition(sf::Vector2f(0, -500));
            Explosion exp(3.0, gs.assets.explosion_texture, gs.nuke.sprite.getPosition());
            gs.explosions.push_back(exp);
        }

        if(gs.nukeship.lives <= 0){
            if(gs.player.nukes < 1) gs.player.nukes++;
            gs.nukeship.lives = 3;
            gs.existsNukeShip = false;
            gs.nukeSound.play();

            gs.player.score += 50;
        }
    }
}
```

Riorganizzazione file In `assets.hpp` sono state spostate le dichiarazioni dei tipi `sf::Texture` e `SoundBuffer`, in `assets.cpp` le inizializzazioni degli sprite. Sono stati creati in `update` (poi spostati e accorpati a `State`) metodi `erase` per eliminare proiettili, nemici eliminati ed esplosioni scadute. E' stato anche realizzato il metodo `centerOrigin` in `entities.hpp` che centra l'origine degli sprite.

Tappa 14 - Modifica texture, opzione riavvia, alert colpo bonus In questa tappa si sono cambiate le texture e gli spritesheet di alcuni elementi quali il giocatore, i nemici, lo sfondo, nave e colpo bonus, il font. Sfondo e nemici sono stati animati utilizzando dei semplici spritesheet da pochi frame. E' stata inoltre aggiunta l'opzione per riavviare la partita dal menù di pausa, gestita analogamente alle altre due opzioni nel menu precedentemente implementate. Infine, è stata aggiunta un'icona intermittente che compare vicino al contatore degli scudi quando il colpo bonus è disponibile.

Notifica punteggi (Tappa 15)

Nell'ultima tappa è stata riorganizzata la logica di gestione del gameOver e le transizioni, accorpando tutto in updateTransitions. E' stata poi aggiunta una notifica del punteggio guadagnato, che appare in prossimità del bersaglio appena eliminato. I metodi movePlayer, spawnShieldCharger, spawnBonusShip, sono stati spostati nelle rispettive struct, e i metodi ausiliari erase sono stati trasferiti dentro State a partire da Update.

```
FloatingText::FloatingText(const sf::Font& font, const std::string& str, sf::Vector2f pos) :
    text(font)
{
    text.setString(str);
    text.setFillColor(sf::Color(255, 0, 0));
    text.setCharacterSize(36);

    sf::FloatRect bounds = text.getLocalBounds();
    text.setOrigin(sf::Vector2f(bounds.position.x + bounds.size.x / 2.0, bounds.position.y + bounds.size.y / 2.0));
    text.setPosition(pos);
}

bool FloatingText::expired() {
    return clock.getElapsedTime().asSeconds() >= duration;
}

void FloatingText::animate() {
    float elapsed = clock.getElapsedTime().asSeconds();

    //sposta verso alto
    text.move(sf::Vector2f(0.0, -2.0));

    //fade-out
    float alphaRatio = 1.0 - (elapsed / duration); //calcola trasparenza es: elapsed/duration=0.5 allora 1-0.5
    if(alphaRatio < 0.0) alphaRatio = 0.0; //evita val neg

    sf::Color color = text.getFillColor();
    color.a = static_cast<std::uint8_t>(255 * alphaRatio); //modifica trasparenza canale alfa rgba
    text.setFillColor(color);
}

void FloatingText::draw(sf::RenderWindow& window) {
    window.draw(text);
}
```

Nelle funzioni in cui vengono gestite collisioni dei colpi di Player viene aggiunto un oggetto FloatingTexts al relativo array, che viene poi disegnato nella doGraphics. La funzione animate viene chiamata in un'ausiliaria updateFloatingTexts.